Re-Creating Mitigating Object Hallucinations in Large Vision-Language Models through Visual Contrastive Decoding

Maxwell Heefner

https://github.com/maxheef/ECE209_Project1.git

Problem

Large-Vision Language Models (LVLMs) are often susceptible to object hallucinations, generating descriptions of objects that are not present in the input image. This behavior is problematic in applications that depend on factual grounding.

Retraining object detection or multimodal models is computationally expensive and time-consuming, making it impractical as a general solution. This drives the need for a training-free method to reduce hallucinations at inference time. One approach is known as Visual Contrastive Decoding (VCD).

Consider a scenario where an attacker is the LVLM's own language prior. This is an insider attack where the model injects likely, but incorrect objects into responses. In this situation, the attacker's goal is to output plausible but ungrounded tokens, even though it has no access to model internals at inference time. Meaning the model generates ungrounded tokens based on learned patterns, despite any visual evidence.

Background

Visual Contrastive Decoding (VCD)

VCD is defined as:

> "*A simple and training-free method that contrasts output distributions derived from original and distorted visual inputs*" (Leng et al [1]).

The idea is to adding Gaussian noise to an input image and compare the model's output distribution between original and perturbed inputs. The contrast suppresses evidence that is not visually present, thus reducing object hallucinations.

Multi-Frequency Contrastive Decoding (MFCD)

MFCD utilizes VCD, but extends it into the frequency domain:

> "*The principle of MFCD is to contrast the original output distribution with the hallucination distributions generated by the input image with low-frequency information removed and the input image with high-frequency information removed, respectively*" [Liu et al. [2]).

Instead of adding Gaussian noise, MFCD will look in the frequency domain and compare an image to itself through high-pass and low-pass filters. The theory is that by looking through different "lenses" this will alleviate object hallucinations in the LVLM. This enables stronger suppression of hallucinated content by disrupting different types of visual information.

Implementation

To reproduce baseline results (Task A), the original VCD codebase from Leng et al. [1] was used with the LLaVA-1.5 model evaluated on the MSCOCO Val2014 dataset using the POPE benchmark (random and popular splits).

A wrapper script (*Main.ipynb*) was implemented with the following structure:

- Task A (VCD):
  - Uses original scripts (*object_hallucination_vqa_llava.py*, *eval_pope.py*) to evaluate regular decoding vs VCD
- Task B (MFCD):
  - Uses original MFCD scripts (*pope/eval.py*, *processor.py*, *eval_utils.py*) to evaluate MFCD on random and popular splits
- Main Runner
  - *Main.ipynb* initializes the GPU, downloads the MSCOCO dataset, executes both tasks, and generates a comparison results table.

Challenges for this experiment included consistency across hardware. Originally Task A was run on a Colab H100 GPU, but was unavailable for Task B. The code was modified to be included in a single runner (*Main.ipynb*) and included setup logic to configure the GPU and enable separate Conda environments and pinned dependencies. The original author's evaluation scripts were utilized for both VCD and MFCD to ensure consistency.

Experiments were conducted on both a Google Colab H100 and G4 GPU. The runtime environment needed to be re-configured to ensure compatibility with the proper Python, Numpy and Pytorch as depicted in requirements.txt. To maintain version control, the GPU pulled all relevant dependencies and code from GitHub and saved all metrics and variables locally.

The evaluation dataset consisted of the MSCOCO Val2014 split, and the model used was the LLaVA 1.5-7b. For consistency and to ensure POPE sensitivity to prompt phrasing, the message response template from the original project (*"Please answer this question with one word."*) remained unchanged. This will limit the model to answering with binary "yes" or "no". The desired parameters for $Seed, Alpha, Beta,$ and $Noise\ Step$ were taken from Leng et al. [1].

Since VCD was computed directly, the improvement approach for MFCD needed to integrate with the same POPE evaluation setting. This allowed for strict method comparison under identical splits, metrics, and reporting. The parameters utilized were pulled directly from the original VCD and MFCD projects to ensure consistency:

- Model: LLaVA-1.5 7B
- Dataset: MSCOCO Val2014 with POPE splits (random, popular)
- Methods: Regular, VCD, MFCD
- Metrics: Accuracy, Precision, Recall, F1
- Hardware: Colab GPU (Blackwell 6000 G4 / H100 depending on run)
- Decoding params
    - VCD: α = 1.0, β = 0.2, noise_step=500
    - MFCD: α = 1.0, β = 0.3 (high/low frequency filtering)
- Outputs: JSON metrics files + summary table

To ensure reproducibility, dependencies were pinned via *requirements.txt* and separate environments were configured for VCD and MFCD.

In order to calculate accuracy, precision, recall and F1 Score the True Positive (TP), True Negative (TN), False Positive (FP) and False Negative (FN) responses were recorded through each iteration. To calculate the metrics seen in table 1, the following formulas were utilized:

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

$$F1 = 2 * Precision * \frac{Recall}{Precision + Recall}$$

$$Accuracy = \frac{TP + TN}{Total\ Questions}$$

$$Yes\ Proportion = \frac{Yes\ Answers}{Total\ Questions}$$

$$Unknown\ Prop = \frac{Unknown}{Total\ Questions}$$

All code is attached below, along with a README.md that explains how LLaVA 1.5-7b was evaluated on the MSCOCO Val2014 dataset using POPE random and popular splits, with both regular decoding, VCD, and MFCD. The attached code was used to generate the Table 1 results.

Dataset	Setting	Model	Decoding	Accuracy	Precision	Recall	F1 Score	Increase
MS COCO	*Random*	LLaVA 1.5-7b	Regular	0.8287	0.9199	0.7200	0.8078	-
			VCD	0.8557	**0.9369**	0.7627	0.8409	4.1%
			MFCD	**0.8677**	0.8506	**0.8920**	**0.8708**	**7.8%**
	Popular		Regular	0.8107	0.8795	0.7200	0.7918	-
			VCD	**0.8357**	**0.8931**	0.7627	0.8227	3.9%
			MFCD	0.8220	0.7835	**0.8900**	**0.8333**	**5.25%**

Table 1. Results on POPE. Regular decoding denotes direct sampling, whereas VCD refers to sampling from our proposed contrastive distribution p_{vcd} and MFCD applies multi-frequency contrastive decoding to further suppress hallucination-driven tokens. The best performances within each setting are bolded[1].

Discussion

Both the random and popular splits showed increased accuracy when decoded with VCD and MFCD. This is consistent with the original results reported by Leng et al [1]. The largest improvements are seen in recall and F1, suggesting that VCD aids in reducing the number of missed TP. Meaning, that the model will be less likely to deny objects that are present in the image.

POPE is designed to evaluate object hallucinations by asking binary questions (e.g., "*Is there a bird in the image?*") that test susceptibility to the presence of language priors or biases. The difference in these two splits is demonstrated in how negative objects are sampled;

- Random Split: negative objects are sampled uniformly
- Popular Split: negative objects are sampled from more common categories (increasing chances of priors in hallucinations)

Since improvement was seen in both splits, it is implied that VCD aids in not only correctly determining if random objects are present, but also mitigates the effects of priors. Meaning that VCD can demonstrate improvement both visually and against language-based biases.

Comparing VCD to MFCD, the Recall and F1 score of MFCD show the greatest improvements over VCD, however, accuracy and precision show a different story. Precision is significantly higher (~8-10%) for VCD than it is with MFCD. Precision can be thought of as: “when the model says Yes, how often is it correct.” This is most likely because MFCD has a more aggressive approach to object hallucination, thus meaning it creates a noisier environment, resulting in more false positives. VCD is conservative in nature and will have fewer false positives, thus leading to higher precision. The aggressive nature of MFCD improves recall drastically by catching more true positive cases.

Conclusion

Different decoding methods can be utilized to mitigate object hallucination in LVLMs. VCD and MFCD have shown consistent performance improvements over standard decoding. This highlights that training-free contrastive decoding methods are effective tools for reducing

hallucinations, but the decision on which method to use depends on the trade-off between precision and recall

Limitations

VCD is limited to adding gaussian noise to each perturbation of the images in the dataset. However, Leng et al. [1] note that more robust techniques – such as object-level blurring – would result more effective hallucination mitigation. By reproducing the results from Leng et al. [1], it has been demonstrated that VCD shows improvements in mitigating object hallucination, but with more robust techniques, even further improvement should be expected. This was proven by introducing MFCD as discussed by Liu et al. [2]. The MFCD approach showed consistent increase in F1 score over VCD, but also has its limitations. MFCD can only be utilized on LVLMs that process images and text input and cannot be used for video or text information. Additionally, MFCD would still be susceptible to neglect of image information or incorrect user instruction.

References

[1] *Sicong Leng, Hang Zhang, Guanzheng Chen, Xin Li, Shijian Lu, Chunyan Miao, and Lidong Bing. Mitigating Object Hallucinations in Large Vision-Language Models through Visual Contrastive Decoding*

[2] *Bingqian Liu, Fu Zhang*, Guoqing Chen, and Jingwei Cheng. Multi-Frequency Contrastive Decoding: Alleviating Hallucinations for Large Vision-Language Models*

[3] *Jihoon Lee and Min Song. Retrieval Visual Contrastive Decoding to Mitigate Object Hallucinations in Large Vision-Language Models.*

README.MD

Selected files

1 printable files

README.md

README.md

Maxwell Heefner

ECE209AS Trustworthy AI

18 Mar 26

Final Project

This folder contains a items for Task A and B that reuses code from `originalProject` and `originalMFCD`

Files

- `/myTasks`: Contains all files written for the completion of Task A and B for the ECE209AS Final Project
  - `Main.ipynd`: Jupyter Notebook that will run all code and configure the Colab GPU for use and then re-create Table 1 from *Mitigating Object Hallucinations in Large Vision-Language Models through Visual Contrastive Decoding* then continue to accomplish improvements to VCD by running a MFCD comparison.
    - Note: All files that the H100 GPU will reference are cloned from GitHub into `/content` storage then writes the outputs to temporary storage on Colab `/content/VCD_project/myTasks/output`
  - `init_setup.py`: Will detect the Colab GPU being used (G4 or H100) and initialize the setup by downloading all requirements listed in `requirements.txt`
  - `analysis.py`: Generates a comparison table for all results of Regular/VCD/MFCD on Regular/Random Splits
  - `sync_repo.py`: Pulls the most recent Git Repository for use on the selected GPU
  - `/scripts`: Contains setup scripts for the Colab G4 or H100 GPU respectively
    - `setup_g4.py`: Will initialize the setup for a G4 GPU
    - `setup_h100.py`: Will initialize the setup for a H100 GPU
  - `/tasks`: Contains scripts to run Task A and Task B requirements
    - `task_a_vcd.py`: Runs 4 iterations (*random*/*popular* and *regular*/*vcd*) and re-creates Table 1 data from: *Mitigating Object Hallucinations in Large Vision-Language Models through*

Visual Contrastive Decoding by *Leng et al*.
    - This file accomplishes this by calling `/eval/object_hallucination_vqa_llava.py` from the original VCD project and implementing all of the desired parameters (*SEED, Alpha, Beta, Noise Step, etc*.).
    - POPE datasets were used from `originalProject` for both random and popular, located here: `experiments/data/POPE/coco/coco_pope_random.json` and `experiments/data/POPE/coco/coco_pope_popular.json`
    - In order to evaluate the results `/eval/eval_pope.py` from `originalProject` was utilized to keep track of True/False Positve/Negative results and calculate *Accuracy*, *Precision*, *Recall* and *F1 Score*
    - VCD source files used by task_a_vcd.py (from originalProject)
    - `/originalProject/experiments/eval/object_hallucination_vqa_llava.py`
    - `/originalProject/experiments/eval/eval_pope.py`
    - `/originalProject/experiments/data/POPE/coco/coco_pope_random.json`
    - `/originalProject/experiments/data/POPE/coco/coco_pope_popular.json`
- `task_b_mfcd.py`: Runs 2 iterations (*random/popular* and *MFCD*) and re-creates Table 3 data from: *Multi-Frequency Contrastive Decoding: Alleviating Hallucinations for Large Vision-Language Models* by *Liu et al*. for comparison with the data found in *Task A*.
    - MFCD source files used by task_b_mfcd.py
        - `/originalMFCD/mfcd/eval_utils/__init__.py`
        - `/originalMFCD/mfcd/eval_utils/eval_utils.py`
        - `/originalMFCD/mfcd/eval/pope/eval.py`
        - `/originalMFCD/mfcd/processor/processor.py`
        - `/originalMFCD/mfcd/utils/` (helper functions used by MFCD evaluation pipeline)
        - `/originalMFCD/mfcd/transformers/` (MFCD-bundled Transformers fork required by evaluation pipeline)

How to Run

Run the `/myTasks/Main.ipynb` all the way through. It will take ~100 min to complete on a colab G4 or H100 GPU. This will synchronize all files from the most recent Git Repository, and generate a neat comparison table for Regular vs. VCD vs MFCD on POPE for both Random and Popular MSCOCO datasets.

Using an Differnt GPUs thorugh Colab

H100 GPU

- Multiple compatibility errors were noted when using the some of the `originalProject` files, because python 3.9 and numpy 1.x must be used

G4 GPU

- Multiple compatibility errors were noted when using the G4 GPU because many of the requirements needed sm_120 and a minimum of Python 3.10

Outputs:

- `/content/VCD_project/myTasks/output`

Documentaion statement

The papers located in `/Papers` were referenced to understand Mitigating Object Hallucinationw with VCD, in addition to the `https://github.com/DAMO-NLP-SG/VCD.git` repository which was cloned into `/originalProject`. For Task B, the Git Repository at: `https://github.com/liubq-dev/mfcd.git` was cloned into `/originalMFCD` for usage in re-creating MFCD results The cloned Git Repository was used to run the experiment, specifically with `/eval/object_hallucination_vqa_llava.py` In addition to the provided material and Git repository, the VSCode Extension of CODEX was utilized to aid with template generation and creation of `setup_g4.py` and `setup_h100.py` to fix compatibility errors with the Colab G4 & H100 GPU respectively. *Prettier* was used to help format and propse commands to generate clean and efficient code.

Citation

@article{leng-etal-2023-vcd, author = {Leng, Sicong and Zhang, Hang and Chen, Guanzheng and Li, Xin and Lu, Shijian and Miao, Chunyan and Bing, Lidong}, title = {Mitigating Object Hallucinations in Large Vision-Language Models through Visual Contrastive Decoding}, journal = {arXiv preprint arXiv:2311.16922}, year = {2023}, url = {https://arxiv.org/abs/2311.16922} }

@inproceedings{liu-etal-2025-mfcd, author = {Liu, Bingqian and Zhang, Fu and Chen, Guoqing and Cheng, Jingwei}, title = {Multi-Frequency Contrastive Decoding: Alleviating Hallucinations for Large Vision-Language Models}, booktitle = {Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing}, year = {2025}, pages = {28568--28584}, address = {Suzhou, China}, publisher = {Association for Computational Linguistics} url = {https://aclanthology.org/2025.emnlp-main.1452.pdf} }

MAIN.IPYNB

Notebook Overview

This notebook will run the full VCD/MFCD Comparison. It initializes the environment, synchronizes the repository, prepares datasets, and runs both Task A (VCD reproduction) and Task B (MFCD improvement) on the MSCOCO POPE splits using LLaVA-1.5. All results are saved to a unified output directory and summarized in a comparison table.

What this notebook does

- Sets up the runtime, installs dependencies, and prepares COCO-Val2014
- Runs Task A (Regular vs. VCD on random/popular POPE)
- Runs Task B (MFCD on random/popular POPE)
- Generates a final comparison table (Regular vs. VCD vs. MFCD)

Outputs

- JSON result files for each split/method
- A formatted table with Accuracy, Precision, Recall, and F1 for all methods

In [1]:
```
import torch
print('Torch:', torch.__version__)
print('CUDA:', torch.cuda.is_available())
if torch.cuda.is_available():
    print('GPU:', torch.cuda.get_device_name(0))
    print('CUDA Version:', torch.version.cuda)
```

```
Torch: 2.10.0+cu128
CUDA: True
GPU: NVIDIA RTX PRO 6000 Blackwell Server Edition
CUDA Version: 12.8
```

In [ ]:
```
import os
from pathlib import Path

# Define paths
REPO_URL = 'https://github.com/maxheef/ECE209_Project1.git'
ROOT = "/content/VCD_project"
MY_TASKS = f"{ROOT}/myTasks"

# Clone if it doesn't exist
if not os.path.exists(ROOT):
    print("Cloning repository for the first time...")
    !git clone {REPO_URL} {ROOT}
else:
```

```
    print("Repository already exists.")

# Move to the scripts for configuration
os.chdir(MY_TASKS)
print(f"Current Directory: {os.getcwd()}")

# Verify files are visible
!ls
```

```
Repository already exists.
Current Directory: /content/VCD_project/myTasks
analysis.py  init_setup.py  __pycache__  sync_repo.py
hidden       Main.ipynb     scripts      tasks
```

Setup

In [35]:
```
# Infrastructure & Data (~8 mins first time, <1 sec after) ---
sys.path.append('/content/VCD_project/myTasks/scripts')
import sys

# Test to find which GPU configurations are available
import importlib.util
print("G4 Setup Found:", importlib.util.find_spec("setup_g4") is not N
print("H100 Setup Found:", importlib.util.find_spec("setup_h100") is n

%run init_setup.py
```

```
G4 Setup Found: True
H100 Setup Found: True
--- Syncing Repository (main) ---
Repository synced successfully.
Conda environments already exist. Skipping hardware setup.
MSCOCO already exists. Skipping download.

[SUCCESS] All components are initialized and quiet.
```

Sync Git

In [60]:
```
# Only need to run this if you do not want to run init_setup.py every
%run sync_repo.py
```

```
--- Syncing Repository (main) ---
Sync complete. Code is up to date.
```

In [37]:
```
# Load bins for Task A and B
from pathlib import Path
VCD_PY = Path('/tmp/vcd_bin.txt').read_text()
MFCD_PY = Path('/tmp/mfcd_bin.txt').read_text()
```

Task A - Reproduction (Midterm)

Reproduce the evaluation setting for:

- **Dataset source**: MSCOCO (POPE split)
- **Model**: LLaVA-1.5
- **Methods**: Regular decoding (baseline) vs. VCD decoding
- **POPE** settings: random and popular

In [38]:
```
# Run Task A
# Compare Regular with VCD on Random/Popular splits
print("Running Task A (VCD)...")
!{VCD_PY} tasks/task_a_vcd.py
```

```
Running Task A (VCD)...

>>> Task A: Running random | regular
/usr/local/miniconda/envs/vcd310/lib/python3.10/site-packages/huggingfa
ce_hub/file_download.py:949: FutureWarning: `resume_download` is deprec
ated and will be removed in version 1.0.0. Downloads always resume when
possible. If you want to force a new download, use `force_download=True
`.
  warnings.warn(
Loading checkpoint shards: 100% 2/2 [00:03<00:00,  1.87s/it]
100% 3000/3000 [04:01<00:00, 12.45it/s]

>>> Task A: Running random | vcd
/usr/local/miniconda/envs/vcd310/lib/python3.10/site-packages/huggingfa
ce_hub/file_download.py:949: FutureWarning: `resume_download` is deprec
ated and will be removed in version 1.0.0. Downloads always resume when
possible. If you want to force a new download, use `force_download=True
`.
  warnings.warn(
Loading checkpoint shards: 100% 2/2 [00:03<00:00,  1.86s/it]
100% 3000/3000 [07:40<00:00,  6.51it/s]

>>> Task A: Running popular | regular
/usr/local/miniconda/envs/vcd310/lib/python3.10/site-packages/huggingfa
ce_hub/file_download.py:949: FutureWarning: `resume_download` is deprec
ated and will be removed in version 1.0.0. Downloads always resume when
possible. If you want to force a new download, use `force_download=True
`.
  warnings.warn(
Loading checkpoint shards: 100% 2/2 [00:03<00:00,  1.85s/it]
100% 3000/3000 [04:00<00:00, 12.48it/s]

>>> Task A: Running popular | vcd
/usr/local/miniconda/envs/vcd310/lib/python3.10/site-packages/huggingfa
ce_hub/file_download.py:949: FutureWarning: `resume_download` is deprec
ated and will be removed in version 1.0.0. Downloads always resume when
possible. If you want to force a new download, use `force_download=True
`.
  warnings.warn(
Loading checkpoint shards: 100% 2/2 [00:03<00:00,  1.85s/it]
100% 3000/3000 [07:39<00:00,  6.52it/s]
```

Task B - Improvement beyond VCD w/ MFCD (Final)

Identify and attempt at least one method that extends or improves VCD and aims to further increase performance (e.g., accuracy) on the same evaluation setting(s) you used in Task A.

MFCD (Multi-Frequency Contrastive Decoding, EMNLP 2025) was utilized for improvement

In [14]:
```
# Run Task B
# Compute MFCD on Random/Popular splits
print("Running Task B (MFCD)...")
!{MFCD_PY} tasks/task_b_mfcd.py
```

```
Running Task B (MFCD)...
processor_config.json: 100% 173/173 [00:00<00:00, 2.82MB/s]
chat_template.json: 100% 701/701 [00:00<00:00, 14.7MB/s]
chat_template.jinja: 100% 674/674 [00:00<00:00, 14.0MB/s]
preprocessor_config.json: 100% 505/505 [00:00<00:00, 10.4MB/s]
tokenizer_config.json: 1.45kB [00:00, 16.9MB/s]
tokenizer.model: 100% 500k/500k [00:00<00:00, 1.22MB/s]
tokenizer.json: 3.62MB [00:00, 73.7MB/s]
added_tokens.json: 100% 41.0/41.0 [00:00<00:00, 851kB/s]
special_tokens_map.json: 100% 552/552 [00:00<00:00, 11.6MB/s]
Applying chat template: 100% 188/188 [00:05<00:00, 35.98it/s]
Processing batch data:  83% 156/188 [00:18<00:03, 10.53it/s]/usr/local/
miniconda/envs/mfcd310/lib/python3.10/site-packages/joblib/externals/lo
ky/process_executor.py:752: UserWarning: A worker stopped while some jo
bs were given to the executor. This can be caused by a too short worker
timeout or by a memory leak.
  warnings.warn(
Processing batch data: 100% 188/188 [00:23<00:00,  8.00it/s]
config.json: 100% 950/950 [00:00<00:00, 14.1MB/s]
model.safetensors.index.json: 70.1kB [00:00, 389MB/s]
Fetching 3 files:   0% 0/3 [00:00<?, ?it/s]
model-00003-of-00003.safetensors:   0% 0.00/4.18G [00:00<?, ?B/s]

model-00002-of-00003.safetensors:   0% 0.00/4.96G [00:00<?, ?B/s]


model-00001-of-00003.safetensors:   0% 0.00/4.99G [00:00<?, ?B/s]
model-00003-of-00003.safetensors:   0% 0.00/4.18G [00:00<?, ?B/s]

model-00002-of-00003.safetensors:   0% 0.00/4.96G [00:00<?, ?B/s]


model-00001-of-00003.safetensors:   0% 0.00/4.99G [00:00<?, ?B/s]
model-00003-of-00003.safetensors:   0% 0.00/4.18G [00:00<?, ?B/s]

model-00002-of-00003.safetensors:   0% 0.00/4.96G [00:00<?, ?B/s]


model-00001-of-00003.safetensors:   0% 0.00/4.99G [00:00<?, ?B/s]
model-00003-of-00003.safetensors:   0% 0.00/4.18G [00:00<?, ?B/s]

model-00002-of-00003.safetensors:   0% 0.00/4.96G [00:00<?, ?B/s]


model-00001-of-00003.safetensors:   0% 0.00/4.99G [00:00<?, ?B/s]
model-00003-of-00003.safetensors:   0% 0.00/4.18G [00:00<?, ?B/s]

model-00002-of-00003.safetensors:   0% 0.00/4.96G [00:00<?, ?B/s]
```

```
model-00001-of-00003.safetensors:   0% 54.1k/4.99G [00:00<5:06:15, 272k
B/s]
model-00003-of-00003.safetensors:   0% 0.00/4.18G [00:01<?, ?B/s]

model-00002-of-00003.safetensors:   0% 0.00/4.96G [00:01<?, ?B/s]
model-00003-of-00003.safetensors:   0% 0.00/4.18G [00:01<?, ?B/s]

model-00002-of-00003.safetensors:   0% 0.00/4.96G [00:01<?, ?B/s]
model-00003-of-00003.safetensors:   0% 0.00/4.18G [00:01<?, ?B/s]

model-00002-of-00003.safetensors:   0% 0.00/4.96G [00:01<?, ?B/s]


model-00001-of-00003.safetensors:   0% 186k/4.99G [00:01<6:02:09, 230k
B/s]
model-00003-of-00003.safetensors:   0% 0.00/4.18G [00:01<?, ?B/s]

model-00002-of-00003.safetensors:   1% 67.0M/4.96G [00:01<00:14, 335MB/
s]


model-00001-of-00003.safetensors:   0% 9.59M/4.99G [00:01<06:15, 13.3M
B/s]
model-00003-of-00003.safetensors:   0% 0.00/4.18G [00:01<?, ?B/s]


model-00001-of-00003.safetensors:   0% 12.6M/4.99G [00:01<06:00, 13.8M
B/s]
model-00003-of-00003.safetensors:   0% 0.00/4.18G [00:02<?, ?B/s]


model-00001-of-00003.safetensors:   0% 17.3M/4.99G [00:02<04:59, 16.6M
B/s]
model-00003-of-00003.safetensors:   0% 0.00/4.18G [00:02<?, ?B/s]


model-00001-of-00003.safetensors:   0% 24.3M/4.99G [00:02<03:45, 22.0M
B/s]
model-00003-of-00003.safetensors:   8% 335M/4.18G [00:02<00:02, 1.68GB/
s]

model-00002-of-00003.safetensors:   3% 134M/4.96G [00:02<00:39, 121MB/
s]
model-00003-of-00003.safetensors:  11% 469M/4.18G [00:02<00:03, 1.08GB/
s]

model-00002-of-00003.safetensors:   5% 256M/4.96G [00:02<00:20, 235MB/
s]

model-00002-of-00003.safetensors:   7% 335M/4.96G [00:02<00:16, 275MB/
```

```
s]


model-00001-of-00003.safetensors:   1% 67.2M/4.99G [00:02<01:39, 49.5M
B/s]
model-00003-of-00003.safetensors:  14% 604M/4.18G [00:03<00:05, 615MB/
s]


model-00001-of-00003.safetensors:   5% 256M/4.99G [00:03<00:20, 237MB/
s]
model-00003-of-00003.safetensors:  16% 671M/4.18G [00:03<00:06, 529MB/
s]

model-00002-of-00003.safetensors:  12% 604M/4.96G [00:03<00:09, 440MB/
s]
model-00003-of-00003.safetensors:  19% 805M/4.18G [00:03<00:05, 572MB/
s]


model-00001-of-00003.safetensors:   7% 374M/4.99G [00:03<00:17, 259MB/
s]
model-00003-of-00003.safetensors:  21% 872M/4.18G [00:03<00:06, 500MB/
s]

model-00002-of-00003.safetensors:  18% 872M/4.96G [00:03<00:09, 443MB/
s]


model-00001-of-00003.safetensors:  12% 605M/4.99G [00:03<00:11, 370MB/
s]
model-00003-of-00003.safetensors:  26% 1.07G/4.18G [00:04<00:06, 501MB/
s]


model-00001-of-00003.safetensors:  14% 675M/4.99G [00:04<00:11, 366MB/
s]
model-00003-of-00003.safetensors:  29% 1.21G/4.18G [00:04<00:05, 544MB/
s]

model-00002-of-00003.safetensors:  19% 939M/4.96G [00:04<00:11, 358MB/
s]


model-00001-of-00003.safetensors:  15% 768M/4.99G [00:04<00:10, 388MB/
s]

model-00002-of-00003.safetensors:  22% 1.07G/4.96G [00:04<00:09, 415MB/
s]


model-00001-of-00003.safetensors:  18% 877M/4.99G [00:04<00:09, 425MB/
```

```
s]
model-00003-of-00003.safetensors:  35% 1.48G/4.18G [00:04<00:04, 597MB/
s]

model-00002-of-00003.safetensors:  26% 1.27G/4.96G [00:04<00:06, 536MB/
s]


model-00001-of-00003.safetensors:  21% 1.02G/4.99G [00:04<00:07, 505MB/
s]
model-00003-of-00003.safetensors:  37% 1.54G/4.18G [00:07<00:21, 121MB/
s]

model-00002-of-00003.safetensors:  27% 1.34G/4.96G [00:07<00:32, 111MB/
s]

model-00002-of-00003.safetensors:  46% 2.28G/4.96G [00:16<00:25, 106MB/
s]
model-00003-of-00003.safetensors:  37% 1.54G/4.18G [00:18<00:21, 121MB/
s]


model-00001-of-00003.safetensors:  21% 1.04G/4.99G [00:18<00:07, 505MB/
s]


model-00001-of-00003.safetensors:  21% 1.04G/4.99G [00:18<03:20, 19.7M
B/s]

model-00002-of-00003.safetensors:  47% 2.35G/4.96G [00:19<00:29, 88.3M
B/s]


model-00001-of-00003.safetensors:  43% 2.15G/4.99G [00:19<00:23, 122MB/
s]
model-00003-of-00003.safetensors:  67% 2.82G/4.18G [00:20<00:12, 106MB/
s]

model-00002-of-00003.safetensors:  53% 2.62G/4.96G [00:20<00:22, 105MB/
s]


model-00001-of-00003.safetensors:  51% 2.55G/4.99G [00:24<00:23, 104MB/
s]

model-00002-of-00003.safetensors:  61% 3.02G/4.96G [00:25<00:19, 99.2M
B/s]
model-00003-of-00003.safetensors:  85% 3.55G/4.18G [00:28<00:06, 98.0M
B/s]

model-00002-of-00003.safetensors:  72% 3.55G/4.96G [00:31<00:15, 93.4M
B/s]
```

```
model-00001-of-00003.safetensors:  69% 3.42G/4.99G [00:33<00:16, 98.2M
B/s]
model-00003-of-00003.safetensors: 100% 4.18G/4.18G [00:35<00:00, 118MB/
s]


model-00002-of-00003.safetensors:  87% 4.29G/4.96G [00:40<00:07, 88.8M
B/s]


model-00001-of-00003.safetensors:  89% 4.43G/4.99G [00:41<00:05, 110MB/
s]

model-00002-of-00003.safetensors: 100% 4.96G/4.96G [00:43<00:00, 115MB/
s]



model-00001-of-00003.safetensors: 100% 4.99G/4.99G [00:44<00:00, 112MB/
s]
Fetching 3 files: 100% 3/3 [00:44<00:00, 14.85s/it]
Loading checkpoint shards: 100% 3/3 [00:00<00:00,  7.97it/s]
generation_config.json: 100% 141/141 [00:00<00:00, 2.03MB/s]
Generating responses: 100% 188/188 [11:47<00:00,  3.77s/it]
Applying chat template: 100% 188/188 [00:04<00:00, 38.66it/s]
Processing batch data: 100% 188/188 [00:21<00:00,  8.76it/s]
Loading checkpoint shards: 100% 3/3 [00:00<00:00,  5.30it/s]
Generating responses: 100% 188/188 [11:33<00:00,  3.69s/it]
```

Show Comparison Table

In [62]:

```python
# Generate A Comparison Table for Regular, VCD, and MFCD
# Code for comparison table is in myTasks/analysis.py
import importlib
import analysis

OUT_DIR = '/content/VCD_project/output'
analysis.show_comparison_table(OUT_DIR)
```

```
/content/VCD_project/myTasks/analysis.py:73: FutureWarning: Setting an
item of incompatible dtype is deprecated and will raise an error in a f
uture version of pandas. Value '-' has dtype incompatible with float64,
please explicitly cast to a compatible dtype first.
  df.at[reg_idx[0], 'Gain vs Regular (%)'] = '-'
```

VQA Mitigation Comparison: Regular vs VCD vs MFCD

Split	Method	Accuracy	Precision	Recall	F1	Gain vs Regular (%)
random	Regular	0.8287	0.9199	0.7200	0.8078	-
random	VCD	0.8557	0.9369	0.7627	0.8409	**+4.10%**
random	MFCD	0.8677	0.8506	0.8920	0.8708	**+7.80%**
popular	Regular	0.8107	0.8795	0.7200	0.7918	-
popular	VCD	0.8357	0.8931	0.7627	0.8227	**+3.91%**
popular	MFCD	0.8220	0.7835	0.8900	0.8333	**+5.25%**

INIT_SETUP.PY

myTasks/init_setup.py

```python
import os
import subprocess
import torch
import sys
import shutil
from pathlib import Path

# Configuration
ROOT = '/content/VCD_project'
REPO_URL = 'https://github.com/maxheef/ECE209_Project1.git'
BRANCH = 'main'
CONDA_BASE = '/usr/local/miniconda'

def run_cmd(cmd, cwd=None, quiet=True):
    """
    Executes shell commands.
    If quiet=True, output is suppressed unless an error occurs.
    """
    stdout_dest = subprocess.DEVNULL if quiet else subprocess.PIPE

    # Use capture_output=True if quiet to store logs in case of failure
    if quiet:
        result = subprocess.run(cmd, shell=True, stdout=stdout_dest, stderr=stdout_dest, env=os.environ.copy(), cwd=cwd)
    else:
        # Stream output in real-time if not quiet
        process = subprocess.Popen(cmd, shell=True, stdout=subprocess.PIPE, stderr=subprocess.STDOUT, text=True, env=os.environ.copy(), cwd=cwd)
        for line in process.stdout:
            print(line, end='')
        process.wait()
        result = process

    if result.returncode != 0:
        raise Exception(f"Command failed: {cmd}")

def setup_repo():
    """
    Pulls the latest code from the specified GitHub repository and branch.
    If the repository already exists, it performs a hard reset to the latest commit.
    """
    print(f"--- Syncing Repository ({BRANCH}) ---")
```

```python
    if os.path.exists(f"{ROOT}/.git"):
        # Silence git fetch/reset
        run_cmd(f"git -C {ROOT} fetch --depth 1 origin {BRANCH}", quiet=True)
        run_cmd(f"git -C {ROOT} reset --hard FETCH_HEAD", quiet=True)
    else:
        if os.path.exists(ROOT): shutil.rmtree(ROOT)
        run_cmd(f"git clone --depth 1 --branch {BRANCH} {REPO_URL} {ROOT}", quiet=True)

    if ROOT not in sys.path: sys.path.append(ROOT)
    print("Repository synced successfully.")

def setup_hardware():
    """
    Detect GPU and set up environments accordingly.
    With Python 3.10 and pinned versions for compatibility.
    """
    vcd_env_path = f"{CONDA_BASE}/envs/vcd310"
    if os.path.exists(vcd_env_path):
        print("Conda environments already exist. Skipping hardware setup.")
        return

    gpu_name = torch.cuda.get_device_name(0) if torch.cuda.is_available() else "CPU"
    print(f"--- Detected {gpu_name}: Installing Environments (this takes ~5 min) ---")

    if "H100" in gpu_name:
        from scripts.setup_h100 import setup_h100_env
        setup_h100_env(root_dir=ROOT)
    else:
        from scripts.setup_g4 import setup_environments
        setup_environments()
    print("Hardware environments installed.")

def setup_datasets():
    """
    Download the MSCOCO validation set if not already present.
    This is required for both VCD and MFCD tasks.
    """
    DATASET_DIR = "/content/datasets/coco"
    VAL_DIR = os.path.join(DATASET_DIR, "val2014")
    ZIP_FILE = os.path.join(DATASET_DIR, "val2014.zip")
    ZIP_URL = "http://images.cocodataset.org/zips/val2014.zip"
```

```python

    if os.path.exists(VAL_DIR):
        print("MSCOCO already exists. Skipping download.")
        return

    print("Downloading MSCOCO (~1GB)... (approx. 2 min)")
    os.makedirs(DATASET_DIR, exist_ok=True)

    run_cmd(f"wget -q -c {ZIP_URL} -O {ZIP_FILE}", quiet=True)

    print("Unzipping dataset... (approx. 2 min)")
    run_cmd(f"unzip -q {ZIP_FILE} -d {DATASET_DIR}", quiet=True)
    os.remove(ZIP_FILE)
    print("Dataset extracted and ready.")

if __name__ == "__main__":
    setup_repo()
    setup_hardware()
    setup_datasets()
    print("\n[SUCCESS] All components are initialized and quiet.")

```

SETUP_G4.PY

myTasks/scripts/setup_g4.py

```python
import os
import subprocess
import shutil
import torch
from pathlib import Path

def run_cmd(cmd, quiet=True):
    """ Utility function to run shell commands with optional quiet mode."""
    if quiet:
        # Run silently in the background
        result = subprocess.run(
            cmd, shell=True, stdout=subprocess.DEVNULL,
            stderr=subprocess.PIPE, text=True, env=os.environ.copy()
        )
        if result.returncode != 0:
            # If it fails, print the error
            print(f"\nError details: {result.stderr}")
            raise Exception(f"Command failed: {cmd}")
    else:
        # Standard streaming output
        process = subprocess.Popen(cmd, shell=True, stdout=subprocess.PIPE, stderr=subprocess.STDOUT, text=True, env=os.environ.copy())
        for line in process.stdout:
            print(line, end='')
        process.wait()
        if process.returncode != 0:
            raise Exception(f"Command failed: {cmd}")

def setup_environments():
    # Configuration
    CONDA_PATH = '/usr/local/miniconda'
    PROJECT_DIR = '/content/VCD_project'
    REQ_FILE = f'{PROJECT_DIR}/requirements.txt'

    VCD_ENV = 'vcd310'
    MFCD_ENV = 'mfcd310'
    VCD_PY = f'{CONDA_PATH}/envs/{VCD_ENV}/bin/python'
    VCD_PIP = f'{CONDA_PATH}/envs/{VCD_ENV}/bin/pip'
    MFCD_PY = f'{CONDA_PATH}/envs/{MFCD_ENV}/bin/python'
    MFCD_PIP = f'{CONDA_PATH}/envs/{MFCD_ENV}/bin/pip'

    # Setup Miniconda
```

```python
    if not os.path.exists(CONDA_PATH):
        print("Installing Miniconda...")
        run_cmd('wget -q https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh -O /tmp/miniconda.sh')
        run_cmd(f'bash /tmp/miniconda.sh -b -p {CONDA_PATH}')

    conda_bin = f'{CONDA_PATH}/bin/conda'
    os.environ['CONDA_PLUGINS_AUTO_ACCEPT_TOS'] = 'yes'
    run_cmd(f"{conda_bin} tos accept --override-channels --channel https://repo.anaconda.com/pkgs/main")
    run_cmd(f"{conda_bin} tos accept --override-channels --channel https://repo.anaconda.com/pkgs/r")

    # Create envs
    for env_name in [VCD_ENV, MFCD_ENV]:
        env_path = f'{CONDA_PATH}/envs/{env_name}'
        if os.path.exists(env_path):
            shutil.rmtree(env_path)
        print(f'Creating environment: {env_name}...')
        run_cmd(f"{conda_bin} create -n {env_name} python=3.10 -y")

    # Install base requirements
    if not os.path.exists(REQ_FILE):
         raise FileNotFoundError(f"Missing {REQ_FILE}")

    print("Installing base requirements (this takes ~3 mins)...")
    for pip_bin in [VCD_PIP, MFCD_PIP]:
        run_cmd(f"{pip_bin} install --upgrade pip setuptools wheel")
        run_cmd(f"{pip_bin} install --no-cache-dir -r {REQ_FILE}")

    # Pin versions
    print("Applying version pins for VCD and MFCD...")
    run_cmd(f"{VCD_PIP} install --no-cache-dir 'transformers==4.31.0' 'tokenizers==0.13.3' 'numpy<2'")
    run_cmd(f"{MFCD_PIP} install --no-cache-dir 'tokenizers==0.21.0' 'numpy<2'")

    # Save binary paths (legacy + current)
    Path('/tmp/vcd_py_path.txt').write_text(VCD_PY)
    Path('/tmp/mfcd_py_path.txt').write_text(MFCD_PY)
    Path('/tmp/vcd_bin.txt').write_text(VCD_PY)
    Path('/tmp/mfcd_bin.txt').write_text(MFCD_PY)

    print(f"G4 Environment Setup Complete.")

```

```python
if __name__ == '__main__':
    setup_environments()

```

SETUP_H100.PY

myTasks/scripts/setup_h100.py

```python
import os
import subprocess
import shutil
import torch
from pathlib import Path

def run_cmd(cmd, cwd=None):
    """Executes shell commands and streams output."""
    process = subprocess.Popen(
        cmd, shell=True, stdout=subprocess.PIPE, stderr=subprocess.STDOUT,
        text=True, env=os.environ.copy(), cwd=cwd
    )
    for line in process.stdout:
        print(line, end='')
    process.wait()
    if process.returncode != 0:
        raise Exception(f"Command failed: {cmd}")

def setup_h100_env(root_dir="/content/VCD_project", env_name="vcd39"):
    orig_dir = os.path.join(root_dir, "originalProject")
    req_file = os.path.join(root_dir, "requirements.txt")
    conda_path = "/usr/local/miniconda"
    conda_bin = f"{conda_path}/bin/conda"
    env_path = f"{conda_path}/envs/{env_name}"
    python_bin = f"{env_path}/bin/python"
    pip_bin = f"{env_path}/bin/pip"

    # Validation
    if not os.path.isdir(orig_dir):
        raise FileNotFoundError(f"Missing originalProject directory: {orig_dir}")

    # Install Miniconda if missing
    if not os.path.exists(conda_bin):
        print("Installing Miniconda...")
        run_cmd('wget -q https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh -O /tmp/miniconda.sh')
        run_cmd(f'bash /tmp/miniconda.sh -b -p {conda_path}')

    # Initialize Conda settings
    run_cmd(f"{conda_bin} config --set always_yes yes --set changeps1 no")
    # Accept TOS if needed
    for channel in ["main", "r"]:
```

```
        run_cmd(f"{conda_bin} tos accept --override-channels --channel https://repo.anaconda.com/pkgs/{channel} || true")


    # Create Environment
    if not os.path.exists(env_path):
        print(f"Creating environment {env_name} (Python 3.9)...")
        run_cmd(f"{conda_bin} create -n {env_name} python=3.9 -y")

    # Install Requirements & Hardware-Specific Pins
    print("Installing H100 optimized packages...")
    run_cmd(f"{pip_bin} install --upgrade pip")
    if not os.path.exists(req_file):
        raise FileNotFoundError(f"Missing requirements.txt: {req_file}")
    run_cmd(f"{pip_bin} install -r {req_file}")

    # H100-specific: Newer Torch (cu121) for performance and compatibility
    run_cmd(f"{pip_bin} install --upgrade --index-url https://download.pytorch.org torch==2.2.2 torchvision==0.17.2")

    # ABI compatibility pin
    run_cmd(f"{pip_bin} install --force-reinstall 'numpy<2'")

    # Record Python path for Orchestrator
    Path('/tmp/vcd_bin.txt').write_text(python_bin)

    # Verification
    print("\n--- H100 Verification ---")
    verify_cmd = f"{python_bin} -c 'import torch, numpy; print(\"Torch:\", torch.__version__); print(\"CUDA Available:\", torch.cuda.is_available()); print(\"GPU:\", torch.cuda.get_device_name(0))'"
    run_cmd(verify_cmd)
    print(f"H100 environment setup complete. Python binary at: {python_bin}")

if __name__ == "__main__":
    setup_h100_env()

```

SYNC_REPO.PY

myTasks/sync_repo.py

```python
import os
import subprocess
from pathlib import Path

ROOT = '/content/VCD_project'
BRANCH = os.getenv('BRANCH', 'main')
REPO_URL = os.getenv('REPO_URL', 'https://github.com/maxheef/ECE209_Project1.git')

def sync():
    """
    Syncs the local repository with the latest code from the specified GitHub repository and branch.
    If the repository already exists, it performs a hard reset to the latest commit to ensure a clean state.
    """
    print(f"--- Syncing Repository ({BRANCH}) ---")
    if os.path.exists(f"{ROOT}/.git"):
        # Fast update
        subprocess.run(f"git -C {ROOT} fetch --depth 1 origin {BRANCH}", shell=True, check=True)
        subprocess.run(f"git -C {ROOT} reset --hard FETCH_HEAD", shell=True, check=True)
    else:
        # Fresh clone
        if os.path.exists(ROOT):
            import shutil
            shutil.rmtree(ROOT)
        subprocess.run(f"git clone --depth 1 --branch {BRANCH} {REPO_URL} {ROOT}", shell=True, check=True)

    # Verify critical path
    if not os.path.isdir(f"{ROOT}/originalMFCD/mfcd"):
        print(f"Error: MFCD directory missing in {ROOT}")
        subprocess.run(f"find {ROOT} -maxdepth 3 -type d | head -n 10", shell=True)
        raise FileNotFoundError("Repo sync failed to find MFCD core.")

    print("Sync complete. Code is up to date.")

if __name__ == "__main__":
    sync()
```

TASK_A_VCD.PY

myTasks/tasks/task_a_vcd.py

```python
import os
import sys
import torch
import gc
import subprocess
import json
import re
from pathlib import Path

# Constants
ROOT = '/content/VCD_project'
ORIG = f'{ROOT}/originalProject'
EXP_DIR = f'{ORIG}/experiments'
OUT_DIR = f'{ROOT}/output'
COCO_VAL = '/content/datasets/coco/val2014'
MODEL_PATH = 'liuhaotian/llava-v1.5-7b'
SEED = '55'

def run_vqa_step(split, method):
    os.makedirs(OUT_DIR, exist_ok=True)
    question_file = f"{EXP_DIR}/data/POPE/coco/coco_pope_{split}.json"
    answers_file = f"{OUT_DIR}/ans_{split}_{method}.jsonl"
    metrics_file = f"{OUT_DIR}/metrics_{split}_{method}.json"

    print(f"\n>>> Task A: Running {split} | {method}")

    cmd = [
        sys.executable, "./eval/object_hallucination_vqa_llava.py",
        "--model-path", MODEL_PATH,
        "--question-file", question_file,
        "--image-folder", COCO_VAL,
        "--answers-file", answers_file,
        "--seed", SEED
    ]

    if method == "vcd":
        cmd += ["--use_cd", "--cd_alpha", "1.0", "--cd_beta", "0.2", "--noise_step", "500"]

    os.chdir(EXP_DIR)
    subprocess.run(cmd, check=True)

```

```python
    # Run Eval and capture output to parse metrics
    eval_cmd = [sys.executable, "./eval/eval_pope.py", "--gt_files", question_file, "--gen_files", answers_file]

    result = subprocess.run(eval_cmd, capture_output=True, text=True, check=True)

    # Parse eval output into a clean JSON metrics file
    def grab(key):
        m = re.search(rf"^{key}:\s*([0-9.]+)", result.stdout, re.MULTILINE)
        return float(m.group(1)) if m else None

    metrics = {
        "accuracy": grab("Accuracy"),
        "precision": grab("Precision"),
        "recall": grab("Recall"),
        "f1": grab("F1"),
    }

    Path(metrics_file).write_text(json.dumps(metrics, indent=2))

    gc.collect()
    torch.cuda.empty_cache()

if __name__ == "__main__":
    for split in ['random', 'popular']:
        for method in ['regular', 'vcd']:
            run_vqa_step(split, method)

```

TASK_B_MFCD.PY

myTasks/tasks/task_b_mfcd.py

```python
import os
import sys
import json
import torch
import gc
from pathlib import Path
from PIL import Image
from torch.utils.data import Dataset

# Constants
ROOT = '/content/VCD_project'
ORIG = f'{ROOT}/originalProject'
MFCD_PATH = f'{ROOT}/originalMFCD/mfcd'
COCO_VAL = '/content/datasets/coco/val2014'
MODEL_PATH = 'llava-hf/llava-1.5-7b-hf'
OUT_DIR = f'{ROOT}/output'

# Add MFCD to path
if MFCD_PATH not in sys.path:
    sys.path.insert(0, MFCD_PATH)

from eval_utils import (prepare_for_generate, generate_responses,
                        batch_prepare_data, MODEL_INFOS)
from eval.pope.eval import eval as pope_eval, recorder

class POPELocal(Dataset):
    def __init__(self, json_path, image_root):
        with open(json_path, 'r') as f:
            self.rows = [json.loads(line) for line in f]
        self.image_root = Path(image_root)

    def __len__(self): return len(self.rows)
    def __getitem__(self, idx):
        row = self.rows[idx]
        img = Image.open(self.image_root / row['image']).convert('RGB')
        return {'question': row['text'], 'image': img, 'label': row['label'].lower()}

def run_mfcd(pope_type):
    device = torch.device('cuda:0')

    class Args:
```

```python
        mfc_high_alpha = 1.0; mfc_low_alpha = 1.0; mfc_beta = 0.3
        mfc_high_pass_cutoff = 0.1; mfc_low_pass_cutoff = 0.9

        mfc_filter_type = 'gaussian'; mfc_jsd = False; mfc_entropy = False
        model_name_or_path = MODEL_PATH; model_type = 'llava-1.5'
        num_workers = 4; eval_batch_size = 16; device = 'cuda:0'
        temperature = 1.2; top_p = 1.0; top_k = 50; max_new_tokens = 2048

    args = Args()
    model_info = MODEL_INFOS['llava-1.5']
    processor, gen_config, _ = prepare_for_generate(args=args, model_info=model_info, device=device)

    pope_json = f"{ORIG}/experiments/data/POPE/coco/coco_pope_{pope_type}.json"
    dataset = POPELocal(pope_json, COCO_VAL)
    prompts = batch_prepare_data(args=args, dataset=dataset, processor=processor,
                                 question_column_name='question', image_column_name='image')

    model = model_info.model_class.from_pretrained(
        MODEL_PATH, torch_dtype=torch.float16, **model_info.model_config
    ).to(device)

    responses = generate_responses(model=model, processor=processor, prompts=prompts, generation_config=gen_config)

    labels = [1 if d['label'] == 'yes' else 0 for d in dataset]
    results = pope_eval(recorder(responses), labels)

    del model, processor
    gc.collect()
    torch.cuda.empty_cache()
    return results

if __name__ == "__main__":
    os.makedirs(OUT_DIR, exist_ok=True)
    for p_type in ['random', 'popular']:
        res = run_mfcd(p_type)
        with open(os.path.join(OUT_DIR, f'metrics_mfcd_{p_type}.json'), 'w') as f:
            json.dump(res, f, indent=2)

```

ANALYSIS.PY

myTasks/analysis.py

```python
import json
import pandas as pd
from pathlib import Path
from IPython.display import display, Markdown

def parse_metrics_json(path: Path):
    """
    Metrics had different formats;
    This will help with the LLava/VCD/MFCD formats.
    """
    if not path.exists():
        return None
    try:
        raw_data = json.loads(path.read_text())

        # Some scripts wrap metrics in a 'metrics' or 'results' key
        data = raw_data.get('metrics', raw_data.get('results', raw_data))

        # Standardize keys (looking for 'accuracy', 'Accuracy', etc.)
        def get_val(keys):
            for k in keys:
                if k in data: return float(data[k])
            return 0.0

        return {
            'Accuracy':  get_val(['accuracy', 'Accuracy', 'acc']),
            'Precision': get_val(['precision', 'Precision', 'prec']),
            'Recall':    get_val(['recall', 'Recall']),
            'F1':        get_val(['f1_score', 'f1', 'F1', 'f1-score'])
        }
    except Exception as e:
        print(f"Error parsing {path.name}: {e}")
        return None

def show_comparison_table(output_dir='/content/VCD_project/output'):
    out_dir = Path(output_dir)
    rows = []

    configs = [
        ('random',  'Regular', out_dir / 'metrics_random_regular.json'),
        ('random',  'VCD',     out_dir / 'metrics_random_vcd.json'),
        ('random',  'MFCD',    out_dir / 'metrics_mfcd_random.json'),
```

```python
        ('popular', 'Regular', out_dir / 'metrics_popular_regular.json'),
        ('popular', 'VCD',     out_dir / 'metrics_popular_vcd.json'),
        ('popular', 'MFCD',    out_dir / 'metrics_mfcd_popular.json'),
    ]

    for split, method, path in configs:
        m = parse_metrics_json(path)
        if m and m['Accuracy'] > 0:
            rows.append({'Split': split, 'Method': method, **m})
        else:
            print(f"Skipping {method} ({split}): File missing or empty metrics.")

    if not rows:
        print(f"No valid metric data found in {out_dir}.")
        return

    df = pd.DataFrame(rows)

    # Logic to calculate percentage improvement vs Regular
    df['Gain vs Regular (%)'] = 0.0
    for split in df['Split'].unique():
        base_rows = df[(df['Split'] == split) & (df['Method'] == 'Regular')]
        if base_rows.empty:
            continue
        base_f1 = base_rows.iloc[0]['F1']
        if base_f1 <= 0:
            continue
        for i in df[df['Split'] == split].index:
            curr_f1 = df.loc[i, 'F1']
            df.at[i, 'Gain vs Regular (%)'] = ((curr_f1 - base_f1) / base_f1) * 100
        # Use "-" for the Regular baseline row
        reg_idx = df[(df['Split'] == split) & (df['Method'] == 'Regular')].index
        if len(reg_idx) > 0:
            df.at[reg_idx[0], 'Gain vs Regular (%)'] = '-'
    # Make column object to allow mixed types (string for baseline)
    df['Gain vs Regular (%)'] = df['Gain vs Regular (%)'].astype(object)

    styled_df = df.style.format({
        'Accuracy': '{:.4f}', 'Precision': '{:.4f}', 'Recall': '{:.4f}',
        'F1': '{:.4f}',
        'Gain vs Regular (%)': lambda v: v if isinstance(v, str) else f'{v:+.2f}%'
    }).map(lambda v: f"color: {'green' if isinstance(v, (int, float)) and v > 0 else 'red'}; font-weight: bold" if isinstance(v, (int, float)) else '',
           subset=['Gain vs Regular (%)']).hide(axis='index')

```

```
    display(Markdown("### VQA Mitigation Comparison: Regular vs VCD vs MFCD"))
    display(styled_df)

```